

Declan Sheehan
Dr. Anderson
Cosc320-001
16 October 2019

Lab 6 READ ME

Summary:

This lab was not nearly as time consuming as previous labs, but I did not appreciate the lab's due date placement being where it was. Essentially, all that was needed to be done was to develop a shuffle function along with randomized quick-sort and the hiring problem. In addition; aggregate previous sort functions and time them.

Time Complexity:

	Quicksort	RQuick Sort	Merge Sort	Heap Sort
Best	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Worst	$O(n^2)$	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

If I had to guestimate; I would say it wouldn't take as much more time to double the input. Roughly $n \log n$ times more time to compute.

When doubling data, the test shows that it takes roughly the same amount of time to compute for the 4 sorts. It is very hard to notice the difference, so that is why I am saying it is roughly $n \log n$ more time.

Absolute Timing Scale:

Each of the sort's nearest asymptotic complexity is $O(n \log n)$ with the exception of Quicksort ($O(n \log n) - O(n^2)$). Judging from the data, the time complexity is $O(n \log n)$, since all the sorts take a very quick amount of time to execute. Each algorithm works respectively, so it would be fair to say that the data retrieved is as expected. As for the hiring problem, the number of swaps looks close to $O(\ln(n))$.

Other:

I do not know the wc runtime (in seconds) for the sorts are. I know that if it is $O(n^2)$, then expect to see > 1 second times for values larger than a couple thousand. With this in mind, each sort should run < 1 second for most of them. I have no clue what's happening.

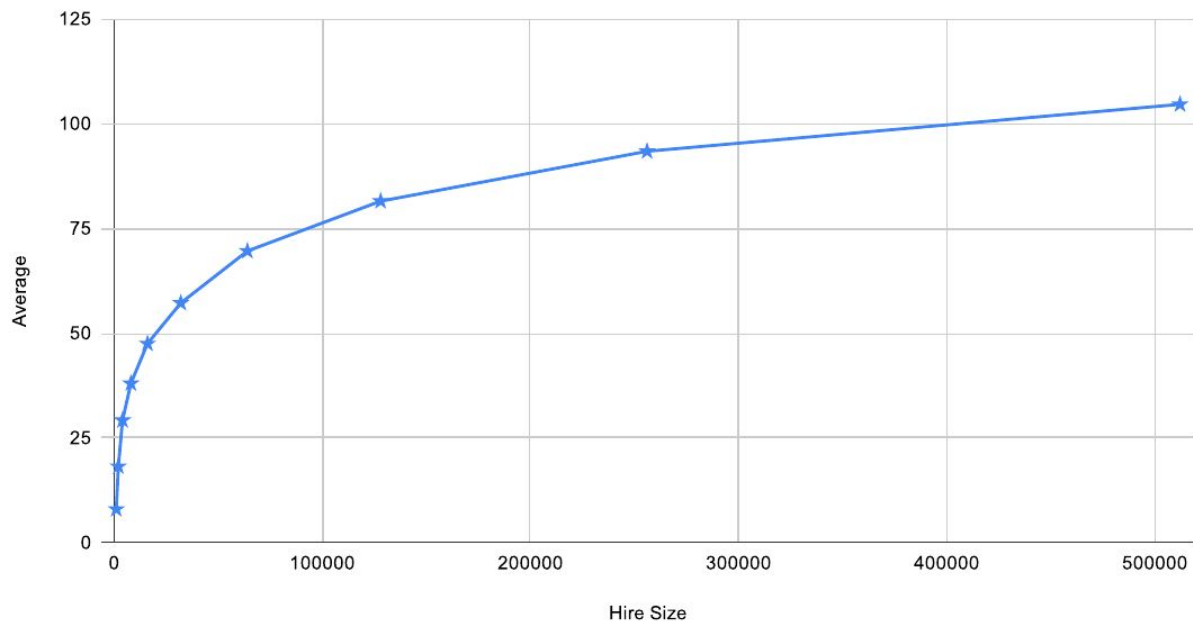
Plots:

Quick Sort, Randomized QS, Merge Sort and Heap Sort



Size	Quick Sort	Randomized QS	Merge Sort	Heap Sort
10000	0.0021377	0.00138987	0.00136326	0.0000911
60000	0.0103931	0.00736831	0.00657691	0.0000918
110000	0.0203465	0.0134532	0.0126558	0.0001007
160000	0.0297884	0.0194699	0.0189365	0.000122
210000	0.0399904	0.0272508	0.0252389	0.001376
260000	0.0510012	0.0344986	0.0314176	0.00127
310000	0.0609631	0.0407171	0.038151	0.00337
360000	0.0729981	0.0475985	0.0447181	0.00338
410000	0.0820641	0.0552185	0.051384	0.00369
460000	0.0935713	0.0642029	0.0577538	0.00493
510000	0.103738	0.0742438	0.0642718	0.00527
560000	0.11442	0.0778068	0.0709717	0.006193
610000	0.127567	0.0840278	0.0786017	0.00647
660000	0.13916	0.0903593	0.0854951	0.007354
710000	0.149688	0.0979282	0.0924094	0.007943
760000	0.166108	0.107615	0.100028	0.009347
810000	0.177557	0.117004	0.106964	0.00962
860000	0.183389	0.124352	0.113468	0.00989
910000	0.197354	0.135474	0.124535	0.0137
960000	0.203313	0.13932	0.127523	0.0189

Number of replacements



Hire Size	Average
1000	8
2000	18.2
4000	29.2
8000	38
16000	47.5
32000	57.2
64000	69.6
128000	81.5
256000	93.4
512000	104.6

Shuffle: How Good Is It?

You can judge the quality of the shuffle by running the hiring algorithm with it. A good shuffle is likely to get you a low number of replacements, as large numbers are tossed up near the front; which reduces the end result. A large replacement number means that there are still an unnatural amount of numbers in order. At least this is what I have observed from messing up my shuffle once.

Code Improvement:

The code could be improved in a fair number of ways. One would be to fix the seg-fault that happens naturally. Another would be to use bigger size-capable data members, e.g. `size_t`, etc.